AIPage: AI-Assisted Page Prediction and Virtual Memory Management System

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Department of Computer Science Engineering

Course Name: Operating Systems and System Programming (OSSP)
(Course Code: 25CS2104E)

Submitted by:

2510030022 - Shrenika Reddy
2510030421 - Aswani Gayatri
2510030393 - A Pavan Reddy
2510030309 - Mohd Abdul Aiyaan

Under the guidance of:

Dr. A. Seenu



Department of Computer Science Engineering
Koneru Lakshmaiah Education Foundation, Aziz Nagar
Aziz Nagar – 500075, Telangana, India

OCT- 2026

TABLE OF CONTENTS — AIPage

1.INTRODUCTION & BACKGROUND ..... Page 3

1.1 Executive Abstract & Project Summary ..... Page 3

1.2 Evolution of Operating Systems and Virtual Memory Management ..... Page 3

1.3 Problem Definition & Challenges in Page Replacement ..... Page 4

1.4 Limitations of Traditional Page Replacement Algorithms ..... Page 4

2. LITERATURE SURVEY & RELATED WORK ..... Page 5

2.1 Comparative Analysis of FIFO, LRU, Optimal and Intelligent Page Replacement … Page 5

3. PROJECT OBJECTIVES & SYSTEM SCOPE ..... Page 6

3.1 Core Technical Contributions & Project Objectives ..... Page 6

3.2 Page Prediction Features and Evaluation Metrics ..... Page 6

4. METHODOLOGY & SYSTEM ARCHITECTURE ..... Page 7

4.1 Multi-Tier Software Architecture ..... Page 7

5. PAGE PREDICTION WORKFLOW & ALGORITHM DESIGN ..... Page 8

5.1 Memory Access Traces, Reuse Patterns and Prediction Features ..... Page 8

5.2 Algorithmic Flowchart & Pseudocode ..... Page 8

6. SYSTEM DESIGN & DATA MANAGEMENT ..... Page 9

6.1 Data Collection, Reuse Statistics and SQLite Database Design ..... Page 9

6.2 Page Replacement Policies and Prediction Evaluation ..... Page 10

7. SYSTEM IMPLEMENTATION & USER INTERFACES ..... Page 11

7.1 AIPage Dashboard and Memory Analysis Interface ..... Page 11

7.2 Page Prediction Features and Replacement Policy Simulation ..... Page 11

7.3 Performance Visualization, Trace Analysis and Result Management ..... Page 12

8. RESULTS AND PERFORMANCE ANALYSIS ..... Page 12

9. CONCLUSION, FUTURE ROADMAP & REFERENCES ..... Page 13

9.1 Project Conclusion & Practical Impact ..... Page 13

9.2 Future Enhancements & Technological Roadmap ..... Page 13

9.3 References ..... Page 14

LIST OF FIGURES

Figure 1: AIPage System Landing Page

Figure 2: AIPage Dashboard for Memory Access Analysis

Figure 3: Memory Access Trace Visualization Interface

Figure 4: AIPage Multi-Tier System Architecture

Figure 5: AI-Assisted Page Prediction Workflow Diagram

Figure 6: Page Replacement Algorithm Flowchart

Figure 7: AIPage Prediction Features — Frequency, Recency and Reuse Distance

Figure 8: Memory Access Trace Collection and Analysis Using Linux `/proc`

Figure 9: AIPage Performance Comparison with FIFO and LRU

Figure 10: Page Hit Rate and Page Fault Analysis

Figure 11: AIPage Prediction Accuracy Evaluation

Figure 12: AIPage Testing and System Evaluation Results

LIST OF TABLES

Table 1: Comparative Analysis of Traditional and Intelligent Page Replacement Algorithms

Table 2: AIPage Architectural Layers and Technology Stack

Table 3: Memory Access Workloads and Frame Capacities Used for Evaluation

Table 4: AIPage Prediction Features and Their Responsibilities

Table 5: Page Replacement Algorithms and Evaluation Metrics

Table 6: AIPage Experimental Results and Performance Comparison with FIFO and LRU

1. INTRODUCTION

1.1 Executive Abstract & Project Summary

Modern operating systems manage physical memory among multiple processes while attempting to minimize page faults and improve overall execution efficiency. When a process accesses a page that is not currently available in physical memory, the operating system must handle a page fault and may need to replace an existing page to make room for the requested one. Traditional page replacement algorithms, such as First-In, First-Out (FIFO) and Least Recently Used (LRU), use predefined replacement rules. However, their effectiveness varies depending on the memory access patterns of an application.

This project presents AIPage: AI-Assisted Page Prediction and Virtual Memory Management System, a user-space prototype designed to analyze memory access traces, identify reuse patterns, and evaluate page prediction and replacement strategies. The system combines frequency-based, recency-based, and reuse-distance-based features to study the likely reuse of memory pages. It provides an interactive web interface for examining memory access behavior and comparing the performance of AIPage with conventional replacement policies.

1.2 Evolution of Operating Systems and Virtual Memory Management

Operating systems have evolved from basic memory allocation mechanisms to sophisticated virtual memory systems that enable multiple processes to execute efficiently within limited physical memory. Virtual memory provides each process with an abstraction of its address space and allows the operating system to manage the relationship between virtual pages and physical memory frames.

As applications became larger and more complex, page replacement algorithms became essential for deciding which pages should be removed when physical memory resources are limited. Early approaches, including FIFO, used page arrival order, while LRU used recent access history to estimate future reuse. The optimal replacement algorithm, commonly known as OPT or MIN, provides a theoretical benchmark by replacing the page whose next use is farthest in the future, although this requires knowledge of future references.

1.3 Problem Definition & Challenges in Page Replacement

Physical memory is limited, and operating systems must decide which resident page to replace when a requested page is absent and no free frame is available. An ineffective replacement decision can remove a page that will soon be accessed again, causing additional page faults and unnecessary memory-management overhead.

Traditional algorithms use fixed strategies that may perform differently across sequential, random, and locality-oriented memory access patterns. FIFO does not consider recent usage, while LRU depends on the assumption that recently accessed pages are more likely to be accessed again. These assumptions may not be equally effective for every workload.

The AIPage project addresses this problem by analyzing historical page access behavior using frequency, recency, last reuse distance, average reuse distance, and reuse count. The system compares its prediction-oriented strategy against conventional policies under controlled workloads and configurable frame capacities. The objective is to investigate whether incorporating multiple reuse-related features can improve page replacement decisions under the evaluated conditions.

1.4 OPERATIONAL VULNERABILITIES OF TRADITIONAL GARAGE MANAGEMENT

The main challenge in page replacement is predicting which resident page is least likely to be needed in the near future. Traditional replacement policies rely on relatively simple decision rules, which may not fully capture the changing behavior of applications.

FIFO and Historical Blindness: FIFO replaces the page that entered memory earliest, regardless of how frequently or recently it has been accessed. This may remove a heavily used page.

LRU and Workload Sensitivity: LRU prioritizes recent usage, but a page that has not been accessed recently may still be important to a future access pattern.

Changing Access Frequencies: Pages can exhibit different access frequencies over time. A policy that ignores frequency may not distinguish frequently reused pages from rarely accessed pages.

Reuse-Distance Estimation: The distance between successive references to a page provides information about its reuse behavior. However, interpreting reuse distance requires considering the workload and available frame capacity.

Prediction and Evaluation Challenges: A page prediction score does not automatically translate into a higher page hit rate. Prediction accuracy and replacement performance must therefore be measured separately.

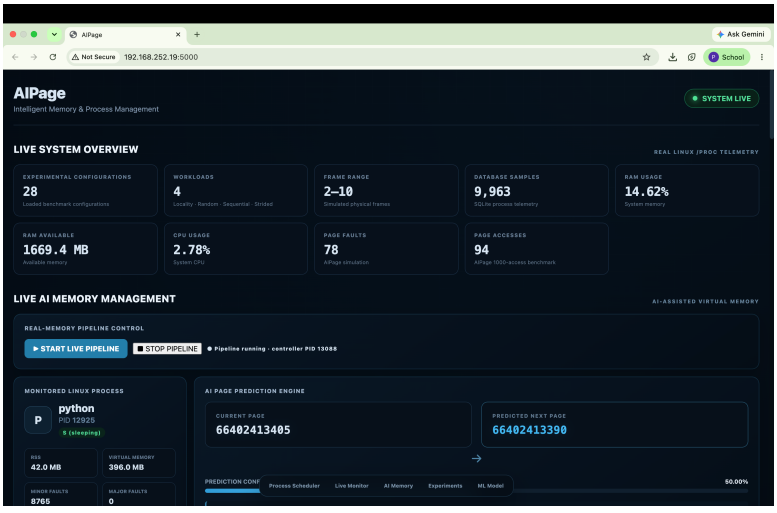


FIGURE1:AIPageSystem Landing Page

2. LITERATURE SURVEY & RELATED WORK

Virtual memory management has been extensively studied because page replacement decisions affect memory utilization, page-fault frequency, and application performance. Traditional algorithms establish the foundations for replacement-policy evaluation, while subsequent research explores adaptive, reuse-aware, and prediction-based approaches.

Belady's optimal replacement algorithm provides a theoretical benchmark by replacing the page whose next access occurs farthest in the future. Since future references are generally unknown during normal execution, practical algorithms approximate this decision using past behavior. FIFO provides a simple baseline, whereas LRU uses recency to make replacement decisions.

Research on adaptive replacement has explored the combination of recency and frequency. Megiddo and Modha's Adaptive Replacement Cache (ARC), for example, adapts between recency and frequency information. LIRS, proposed by Jiang and Zhang, uses inter-reference recency to improve replacement decisions. Other research examines reuse distance and its relationship with memory hierarchy performance.

More recent work has investigated predictive and learning-based approaches to cache and memory management. Such approaches attempt to extract patterns from historical access behavior instead of relying exclusively on a fixed replacement rule. Their performance depends on the quality of training or historical information, the characteristics of the workload, and the overhead introduced by prediction.

AIPage builds on these concepts through a prototype that evaluates frequency, recency, reuse distance, and reuse count features. It compares its implemented prediction-oriented policy with FIFO and LRU across different workload patterns and frame capacities. The system is an educational and experimental prototype rather than a modification of the Linux kernel's production page replacement mechanism.

Table 1:Comparative Analysis of Traditional and Prediction-Oriented Page Replacement Algorithms

Algorithm	Decision basis	Main advantage	Main limitation
FIFO	Page arrival order	Simple to implement	Ignores access frequency and recency
LRU	Most recent access history	Uses temporal locality	May perform poorly when recent usage is not predictive
OPT/MIN	Future reference sequence	Theoretical benchmark	Requires future knowledge
AIPage	Implemented reuse-aware prediction features	Evaluates multiple historical access signals	Effectiveness depends on feature design and workload

3. PROJECT OBJECTIVES & STAKEHOLDER ECOSYSTEM

3.1 Core Technical Contributions & Project Objectives

AIPage aims to provide a practical environment for studying page prediction and replacement strategies using memory access traces. The project combines a web interface, trace analysis, feature calculation, and comparative evaluation to make memory-management experiments easier to observe and understand.

The major objectives are:

- To study the behavior of conventional FIFO and LRU page replacement algorithms.
- To analyze memory access patterns using frequency, recency, and reuse-related information.
- To analyze memory access patterns using frequency, recency, and reuse-related information.
- To implement and evaluate seven prediction features: Frequency-10, Frequency-50, Frequency-100,
- Recency, Last Reuse Distance, Average Reuse Distance, and Reuse Count.
- To provide an interface for inspecting trace information and experimental outcomes.
- To compare page hits, page faults, replacements, and hit rates across different policies.
- To demonstrate memory-management concepts through a user-space prototype without modifying the Linux kernel.

The project focuses on experimental analysis and educational demonstration. Its conclusions are limited to the workloads, traces, policies, and evaluation conditions used in the implementation.

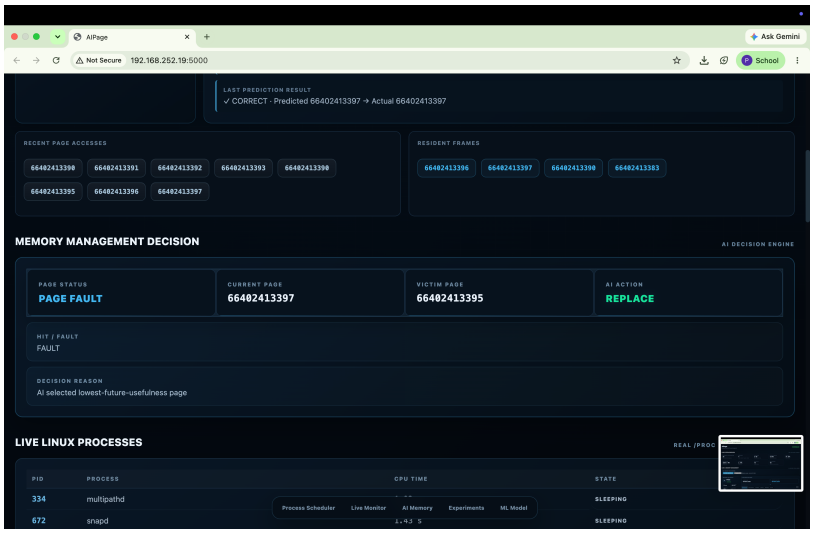


FIGURE 2:AIPage Dashboard for Memory Access Analysis

3.2 Target Users, Prediction Features & Evaluation Scope

AIPage is primarily intended for students, operating systems learners, and developers interested in virtual memory management and page replacement. It helps users examine how historical page accesses can be converted into features and how different replacement policies respond to a given memory reference sequence.

The system uses the following seven features:

1. Frequency-10: Examines access frequency over a recent window of 10 references.
2. Frequency-50: Examines access frequency over a recent window of 50 references.
3. Frequency-100: Examines access frequency over a recent window of 100 references.
4. Recency: Represents how recently a page was accessed.
5. Last Reuse Distance: Represents the distance between the most recent reference and the previous reference to the same page.
6. Average Reuse Distance: Represents the average distance between repeated references to a page.
7. Reuse Count: Records how many times a page has been reused within the evaluated history.

The experimental setup includes locality-oriented, random, sequential, and strided workloads. Frame capacities of 2, 3, 4, 5, 6, 8, and 10 are used in the review's stated evaluation plan. These configurations allow the replacement policies to be compared under different memory constraints.

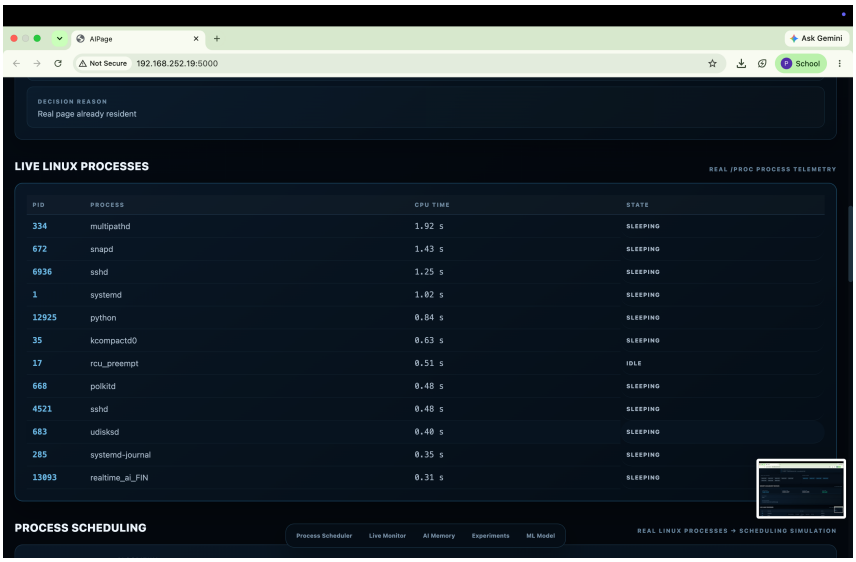


FIGURE 3:Memory Access Trace Visualization Interface

4. METHODOLOGY & SYSTEM ARCHITECTURE

4.1 Multi-Tier Software Architecture

AIPage follows a modular architecture that separates the user interface, backend processing, trace analysis, and data storage. This separation makes it easier to examine each part of the system independently and supports organized development and testing.

Presentation Tier — Web Interface: The frontend uses HTML, CSS, and JavaScript to present the available analysis features and display experimental results. The interface allows users to inspect memory-management information and view performance comparisons.

Application Tier — Python and Flask: The backend processes application requests, coordinates trace analysis, invokes the relevant algorithms, and prepares results for the interface. Flask provides the web application layer.

Algorithm and Analysis Tier — C and Python: The project uses C/GCC and Python for its implementation and analysis tasks. This layer supports page replacement simulation, historical feature calculations, and comparative evaluation according to the implemented code.

Data Persistence Tier — SQLite: SQLite provides local structured storage for the information recorded by the application, according to the implemented database design.

System Information Layer — Linux `/proc`: Linux `/proc` interfaces provide process and system information where used. These interfaces should not be interpreted as a complete source of every virtual-memory reference made by a process.

The architecture connects these components to provide an experimental environment for examining memory access behavior and evaluating replacement policies. The exact responsibility of each component should correspond to the source code and configuration included in the submitted project.

Architectural layer	Technology	Responsibility
Presentation	HTML, CSS, JavaScript	User interface and result visualization
Web application	Python, Flask	Request handling and backend coordination
Algorithm implementation	C/GCC and Python	Trace processing and replacement-policy evaluation
Data storage	SQLite	Local structured data persistence
System information	Linux `/proc`	Process and system information where applicable

TABLE 2:AIPage Architectural Layers and Technology Stack

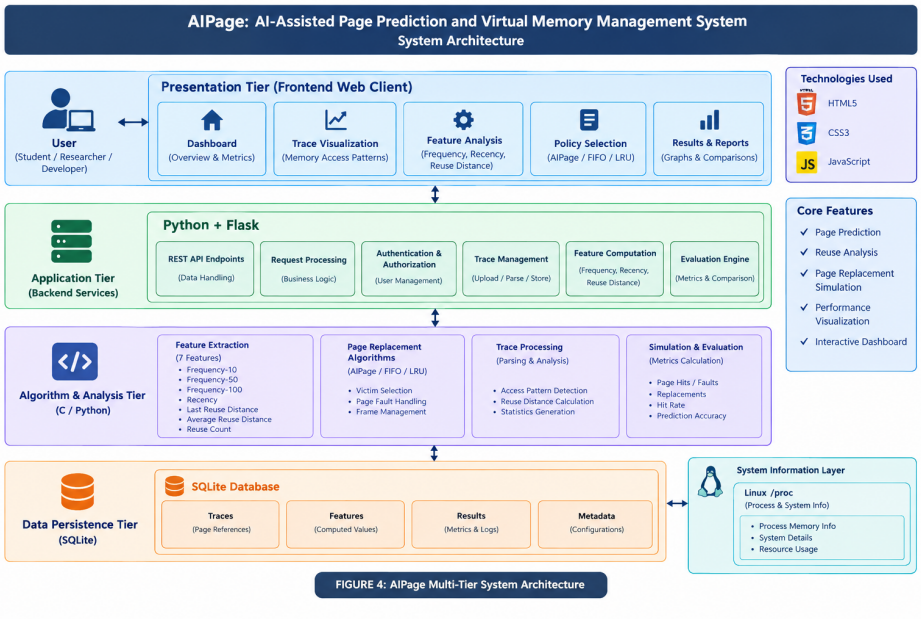


FIGURE 4:AIPage Multi-Tier System Architecture

5. PAGE PREDICTION WORKFLOW & ALGORITHM DESIGN

5.1 Memory Access Traces, Reuse Patterns & Prediction Features

AIPage evaluates page replacement by processing a sequence of memory references. Each reference identifies the page requested at a particular step in the trace. If the requested page is already resident in a frame, the access is recorded as a hit. Otherwise, it is recorded as a miss or page fault in the simulation. When all frames are occupied, the replacement policy selects a resident page for removal.

The prediction-oriented approach examines historical features associated with page accesses. Frequency windows help characterize repeated usage over different observation periods. Recency measures how recently a page was accessed, while reuse distance describes the separation between successive references. Reuse count represents repeated usage over the available history.

These features provide different views of memory locality. Frequency emphasizes repeated activity, recency emphasizes recent use, and reuse distance characterizes the spacing between accesses. Their usefulness depends on the trace and on how the implemented policy combines the feature values.

The experimental workflow evaluates the implemented AIPage strategy alongside FIFO and LRU. Workload types and frame capacities are varied to observe how each policy behaves under different conditions. Prediction accuracy is recorded as a separate measure from the final page hit rate.

Component	Responsibility
Memory access trace	Provides the ordered sequence used in simulation
Frequency features	Characterize access frequency over selected windows
Recency feature	Characterizes how recently a page was accessed
Reuse-distance features	Describe the separation between repeated references
Reuse count	Records repeated page usage
SQLite	Supports local structured persistence as implemented
Evaluation output	Records or presents policy metrics and comparisons

TABLE 3:Memory Access Workloads and Frame Capacities Used for Evaluation

5.2 Algorithmic Flowchart & Pseudocode

General page replacement simulation pseudocode

```
INPUT: reference_sequence, frame_capacity, policy

INITIALIZE empty frames
INITIALIZE hits = 0
INITIALIZE faults = 0
INITIALIZE replacements = 0

FOR each page in reference_sequence:
    IF page is already present in frames:
        hits = hits + 1
        UPDATE access history
    ELSE:
        faults = faults + 1

        IF a free frame is available:
            LOAD page into a free frame
        ELSE:
            victim = SELECT_VICTIM(policy, access_history)
            REMOVE victim from frames
            LOAD page into the available frame
            replacements = replacements + 1

        UPDATE access history

hit_rate = (hits / total_references) * 100

OUTPUT hits, faults, replacements, hit_rate
```

For FIFO, the victim is selected according to arrival order. For LRU, the least recently used resident page is selected. For AIPage, victim selection uses the implemented prediction or feature-scoring logic. This pseudocode describes the general evaluation flow and should not be treated as a verbatim copy of the project's source implementation.

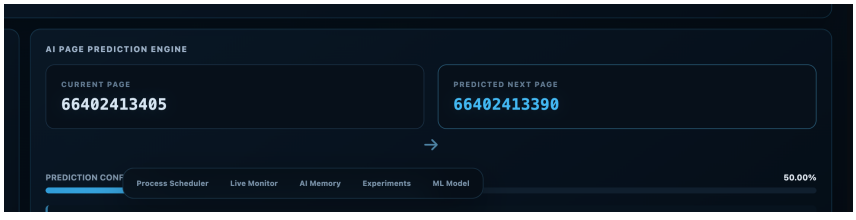


FIGURE 5: AIPage Page Prediction and Replacement Workflow Diagram

6. SYSTEM DESIGN & DATA MANAGEMENT

6.1 Data Collection, Reuse Statistics & SQLite Database Design

AIPage uses memory access information to evaluate how pages are reused over time. The analysis records or derives information such as page references, historical access frequency, recent usage, reuse distance, and replacement outcomes.

The precise tables, columns, keys, and relationships should be documented from the submitted database schema rather than assumed. Similarly, the use of Linux `/proc` provides system information but does not, by itself, establish that the project captures a complete page-reference stream..

Component	Responsibility
Memory access trace	Provides the ordered sequence used in simulation
Frequency features	Characterize access frequency over selected windows
Recency feature	Characterizes how recently a page was accessed
Reuse-distance features	Describe the separation between repeated references
Reuse count	Records repeated page usage
SQLite	Supports local structured persistence as implemented
Evaluation output	Records or presents policy metrics and comparisons

TABLE 4:AIPage Data Components and Responsibilities

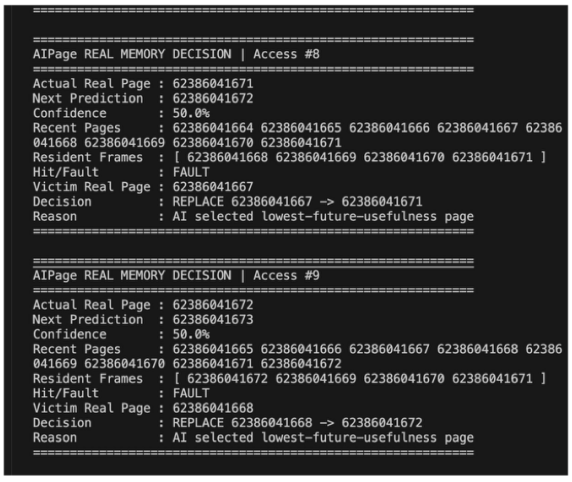


FIGURE 6: AIPage Data Flow and SQLite Storage Design

6.2 Algorithm Selection, Data Integrity & Evaluation Metrics

AIPage compares three replacement strategies: its implemented prediction-oriented policy, FIFO, and LRU. The same reference sequence and frame capacity should be used when comparing policies so that differences in results reflect the replacement decisions rather than inconsistent experimental inputs.

The principal evaluation metrics are:

- Page hits: References satisfied by pages already present in memory.
- Page faults or misses: References for which the requested page is absent.
- Page replacements: Resident pages removed to make space for requested pages.
- Hit rate: Percentage of references satisfied by resident pages.
- Prediction accuracy: Percentage of evaluated predictions that match their corresponding targets.

Policy or metric	Purpose
AIPage	Evaluates the implemented reuse-aware prediction strategy
FIFO	Provides an arrival-order baseline
LRU	Provides a recency-based baseline
Page hits	Measures successful resident-page accesses
Page faults	Measures unsuccessful resident-page accesses
Replacements	Measures resident-page removals
Hit rate	Measures the proportion of references served as hits
Prediction accuracy	Measures correctness over evaluated predictions

TABLE 5: Page Replacement Policies and Evaluation Metrics

7. SYSTEM IMPLEMENTATION & USER INTERFACES

7.1 AIPage Dashboard & Memory Analysis Interface

The AIPage interface provides a visual environment for examining page prediction and replacement-policy behavior. The dashboard is intended to make experimental outputs accessible without requiring users to inspect every algorithmic operation directly in the source code.

The web application uses Flask on the backend and web technologies on the frontend. Depending on the available interface implementation, the dashboard can display trace information, feature values, page replacement outcomes, and comparative performance metrics. The interface should be documented using actual screenshots from the submitted project.

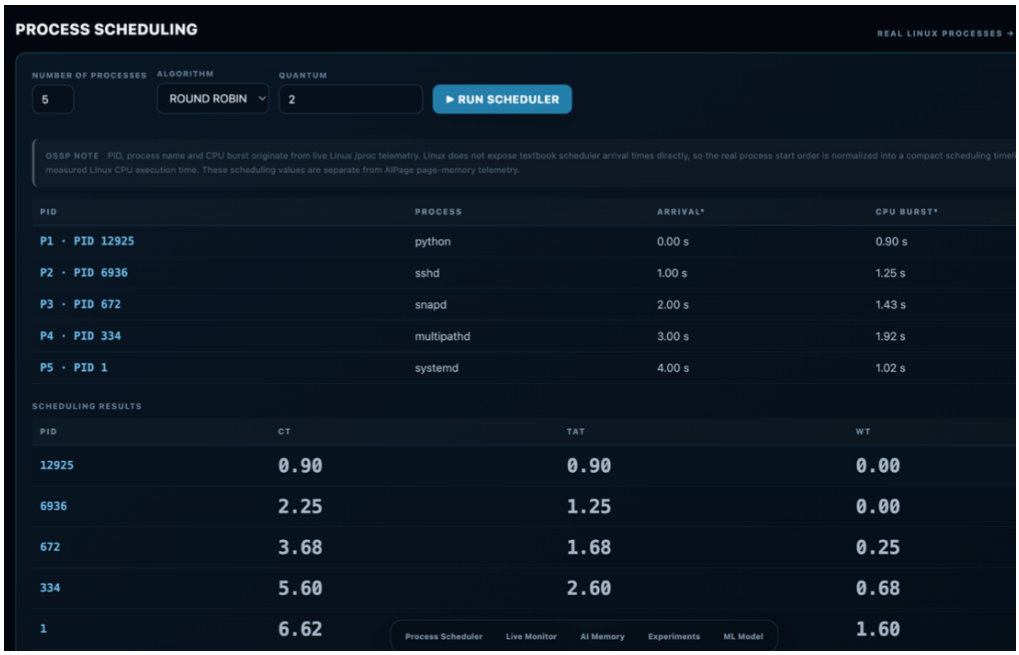


FIGURE 7: AIPage Dashboard with Memory Analysis Results.

7.2 Page Prediction Features & Replacement Policy Simulation

The prediction component analyzes historical page access behavior using the seven features defined in the project. Frequency-10, Frequency-50, and Frequency-100 examine access frequency across different windows. Recency captures the recentness of page access, while last reuse distance, average reuse distance, and reuse count describe repeated access patterns.

The simulation component evaluates memory references against a configurable number of frames. It records whether each access results in a hit or a fault and applies the selected replacement policy when the requested page is absent and the frames are full.

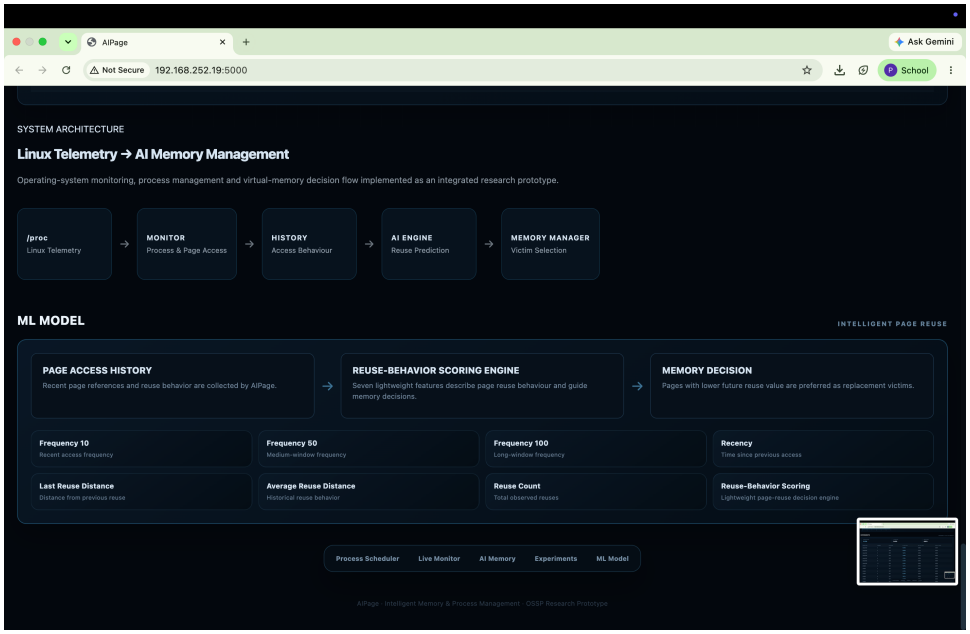


FIGURE 8: AIPage Prediction Features and Replacement Policy Simulation

7.3 Performance Visualization, Trace Analysis & Result Management

AIPage's evaluation component summarizes the results of the replacement-policy experiments. The principal outputs include page hits, faults, replacements, hit rates, and prediction accuracy. Comparing these measures helps identify differences between the policies under the evaluated trace conditions.

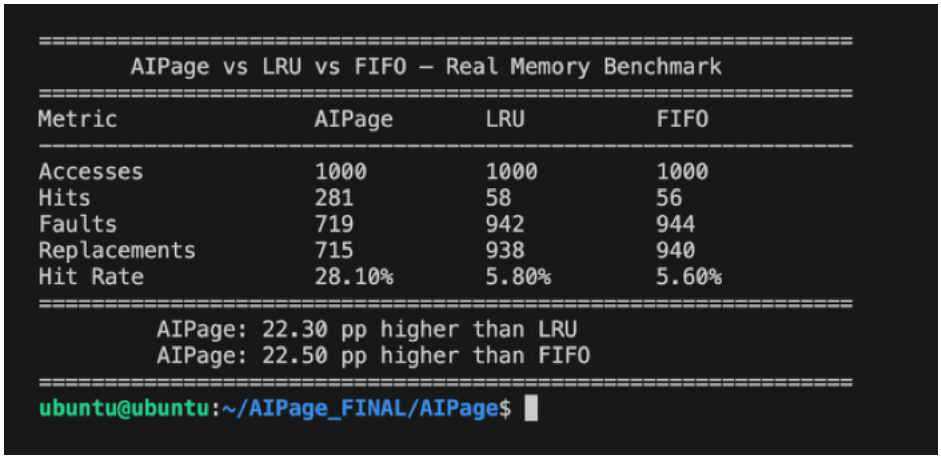


FIGURE 9: AIPage Performance Comparison with FIFO and LRU

8. RESULTS AND PERFORMANCE ANALYSIS

AIPage is evaluated through trace-based experiments that compare the implemented prediction-oriented strategy against FIFO and LRU. The evaluation examines page hits, page faults, replacements, and hit rate. Prediction accuracy is measured separately because it represents the correctness of the prediction process rather than the fraction of memory references served as hits.

In the reported 1,000-access real-memory trace experiment, the following results were recorded

Metric	AIPage	LRU	FIFO
Total references	1,000	1,000	1,000
Page hits	281	58	56
Page faults/misses	719	942	944
Page replacements	715	938	940
Hit rate	28.10%	5.80%	5.60%

TABLE 6: AIPage Experimental Results and Performance Comparison

Metric	AIPage	LRU	FIFO
Accesses	1000	1000	1000
Hits	281	58	56
Faults	719	942	944
Replacements	715	938	940
Hit Rate	**28.10%**	**5.80%**	**5.60%**

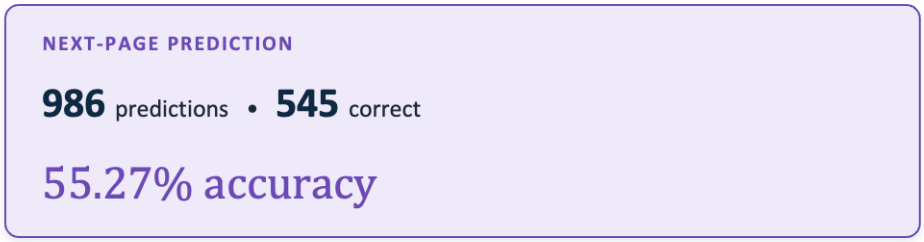


FIGURE 12:AIPage Testing and System Evaluation Results

9. CONCLUSION AND FUTURE WORK & REFERENCES

9.1 Project Conclusion

AI-Assisted Page Prediction and Virtual Memory Management System presents a user-space prototype for studying memory access patterns and evaluating page replacement strategies. The project combines frequency, recency, reuse distance, and reuse count features with a simulation environment that compares the implemented AIPage strategy against FIFO and LRU.

The reported 1,000-reference experiment recorded a 28.10% hit rate for AIPage, compared with 5.80% for LRU and 5.60% for FIFO. A separate prediction evaluation reported 55.27% accuracy across 986 evaluated predictions. These results demonstrate the prototype's ability to measure and compare replacement behavior under the specific conditions tested.

The project also illustrates important distinctions between collecting system information, constructing memory access traces, predicting future accesses, and evaluating replacement outcomes. Since the prototype does not modify the production Linux kernel, its results should be understood as experimental findings rather than direct evidence of improved kernel-level memory management.

9.2 Future Research & Engineering Roadmap

Advanced Prediction Models: Evaluate additional statistical or machine-learning models to determine whether more sophisticated prediction methods improve performance over the existing feature-based strategy.

Expanded Workload Evaluation: Test larger and more diverse memory access traces, including workloads with different locality characteristics, and report results separately for each frame capacity.

Prediction Overhead Analysis: Measure the computational cost and memory overhead of feature extraction and prediction to assess whether any observed improvement justifies the additional processing.

9.3 REFERENCES

[1] L. A. Belady, “A Study of Replacement Algorithms for a Virtual-Storage Computer,” IBM Systems Journal, vol. 5, no. 2, pp. 78–101, 1966.

[2] N. Megiddo and D. S. Modha, “ARC: A Self-Tuning, Low Overhead Replacement Cache,” in Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST), 2003, pp. 115–130.

[3] S. Jiang and X. Zhang, “LIRS: An Efficient Low Inter-Reference Recency Set Replacement Policy to Improve Buffer Cache Performance,” in Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2002, pp. 31–42.

[4] C. Ding and Y. Zhong, “Reuse Distance Analysis,” Technical Report, Department of Computer Science, College of William and Mary, 2003.

[5] G. Keramidas, P. Petoumenos, and S. Kaxiras, “Cache Replacement Based on Reuse-Distance Prediction,” in Proceedings of the 25th International Conference on Computer Design (ICCD), 2007.

[6] A. Jain and C. Lin, “Back to the Future: Leveraging Belady’s Algorithm for Improved Cache Replacement,” in Proceedings of the 43rd Annual International Symposium on Computer Architecture (ISCA), 2016.

[7] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts. Wiley.

[8] Linux Kernel Documentation, “The `/proc` Filesystem.” Available: https://docs.kernel.org/filesystems/proc.html

[9] Python Software Foundation, “Python Documentation.” Available: https://docs.python.org/3/

[10] Pallets Projects, “Flask Documentation.” Available: https://flask.palletsprojects.com/

[11] SQLite Consortium, “SQLite Documentation.” Available: https://www.sqlite.org/docs.html

[12] GNU Project, “GCC Documentation.” Available: https://gcc.gnu.org/onlinedocs/